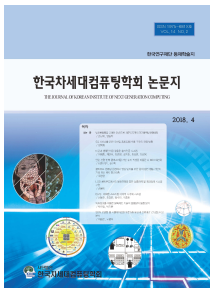
학술논문 검색서비스 earticle



한국차세대컴퓨팅학회 논문지
Vol.14 No.2

ISSN : 1975-681X(Print)

SDN 모델링 및 시뮬레이션을 위한 Mininet과 OPNET 간 성능 비교 분석

이승운, 노병희

To cite this article : 이승운, 노병희 (2018) SDN 모델링 및 시뮬레이션을 위한 Mininet과 OPNET 간 성능 비교 분석, 한국차세대컴퓨팅학회 논문지, 14:2, 82-93

① earticle에서 제공하는 모든 저작물의 저작권은 원저작자에게 있으며, 학술교육원은 각 저작물의 내용을 보증하거나 책임을 지지 않습니다.

② earticle에서 제공하는 콘텐츠를 무단 복제, 전송, 배포, 기타 저작권법에 위반되는 방법으로 이용할 경우, 관련 법령에 따라 민, 형사상의 책임을 질 수 있습니다.

www.earticle.net

Download by : 203.232.XXX.XX1

Accessed : Monday, April 6, 2026 9:54 AM

SDN 모델링 및 시뮬레이션을 위한 Mininet과 OPNET 간 성능 비교 분석

Performance Comparisons between Mininet and OPNET Simulators for SDN Modeling and Simulation

이승운, 노병희[1)]
Seungwoon Lee, Byeong-hee Roh

(16499) 경기도 수원시 영통구 월드컵로 206 아주대학교 컴퓨터공학과
{swleeyg, bhroh}@ajou.ac.kr

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM

요 약

SDN은 네트워크 장비의 제어 기능을 데이터 전달 기능과 분리하고 중앙 관리 시스템에 의해 관리되는 미래 네트워크 기술로 다양한 네트워크 문제를 해결할 수 있다. 이러한 장점으로 SDN 도입은 여러 분야에서 고려되고 있으며, 모의 실험을 위해 주로 Mininet 시뮬레이터가 사용되고 있다. Mininet을 통해 빠르게 SDN 환경을 실험해볼 수 있으나 시스템의 안정성, 신뢰성 검증 실험, 실환경을 모사한 네트워크 실험을 수행하기에는 다소 부족하다. OPNET은 신뢰성이 입증된 네트워크 시뮬레이터로 SDN 모델을 지원하기 시작하였으나 아직 관련 연구가 전무하다. 따라서 본 논문에서는 SDN M&S를 위한 두 시뮬레이터 간의 성능 비교 분석을 수행한다. 먼저 두 시뮬레이터 간 성능지표 비교를 위한 분석 모델을 제안한다. 효율적인 분석을 위하여 해당 모델에서는 토폴로지 모델 변환 방법, 트래픽 생성 모델을 포함한다. 이 분석 모델을 통하여, Mininet과 OPNET에서의 처리량, 단대단 지연시간, 지터 등 성능 지표를 비교하였다. Mininet은 처리량은 잘 반영하나, 지연시간이나 지터와 같이 네트워크 부하에 따른 변화를 제대로 반영하지 못하는 것으로 나타났다. 반면에 OPNET은 실제 네트워크 환경을 반영하고, 정밀한 네트워크 분석 실험이 가능함을 보여주었다.

Abstract

SDN is the future network paradigm of decoupling control plane and data plane by central controller system and helps to solve many networking problems. In general, Mininet is used for SDN simulation. Although it is possible to prototype virtual SDN networks fast with Mininet, it is not for stress testing to validate the stability and reliability and simulation suitable for real environment. OPNET, a well-known network simulator, has also released SDN model, but OPNET based SDN work is not published yet. In this paper, we compare performance parameters between two simulators. First, we propose the Comparative Analysis Model for efficiency including topology model

1) 교신저자

논문투고일자 : 2018. 03. 23 심사일자 : 2018. 04. 11 게재확정일자 : 2018. 04. 25

converter and traffic generation model. Through this model, we compared throughput, End-To-End delay, and jitter between Mininet and OPNET. The Mininet shows the throughput performance very well, while it can not reflect the variations of dealy and jitter performances varying network loads. On the other hand, OPNET can reflect real network operational environements, and it also shows that it is desirable to carry out detail experiments for network analysis.

키워드: 소프트웨어 정의 네트워킹, 모델링 및 시뮬레이션, Mininet, OPNET
Keyword: Software Defined Networking, Modeling and Simulation, Mininet, OPNET

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM

1. 서론

SDN(Software-Defined Networking, 소프트웨어 정의 네트워킹)[1]은 네트워크 장비의 제어 기능을 데이터 전달 기능과 분리하고, 중앙 시스템에 의해 관리되는 미래 네트워크 기술이다. 프로그래밍이 가능한 소프트웨어 기반의 컨트롤러는 연결된 SDN 스위치들의 데이터 패킷을 확인하고 규칙을 생성해 플로우 테이블에 추가한다. 이는 네트워크 트래픽의 중앙 제어 및 관리를 가능하게 하고 기존의 하드웨어 기반의 네트워킹에서의 다양한 문제들을 해결할 수 있다.

SDN이 보유한 장점으로 인해 여러 분야에서 SDN 도입을 위한 연구들이 진행되고 있다. 네트워크를 연구함에 있어서, 성능분석을 위한 실험 평가 환경 수립이 요구된다. 이러한 성능분석은 주로 망의 규모, 장비의 제한, 정립 중인 표준 등의 제약으로 인해 실환경을 모사한 가상 환경에서 모델링 및 시뮬레이션 (M&S, Modeling and Simulation) 방법을 기반으로 수행되어 왔다.

SDN M&S를 위하여 OpenFlow 프로토콜[2]을 바탕으로 오픈소스 도구인 Mininet[3]이 많이 사용되고 있다. Mininet은 SDN 스위치와 호스트 간의 토폴로지 환경을 간단하게 실험하기에 용이하다. 하지만 노드, 링크 모델이 다양하지 못하고 모델의 세부 성능, 명세 등을 설정할 수 없다. 이러한 이유로 네트워킹 시스템의 안정성과 신뢰성을 검증하는 실험이나 현실적인 네트워크 환경이 필요한 실험 등에서 Mininet을 시뮬레이션 도구로 사용하기엔 부족한 면이 있다[4].

통신 네트워크와 연관된 다양한 분야의 M&S를 위하여 활용되고 있는 시뮬레이터로는 OPNET Modeler (Riverbed Modeler, 이하 OPNET) [5]이 있다. OPNET에서는 현재 표준으로 제시된 통신 프로토콜과 네트워킹 기능을 제공하고 있으며, 최근에 SDN 모델도 지원을 시작하였다. OPNET은 실장비의 명세를 반영한 다양한 벤더의 네트워크 모델들을 구현하여 시뮬레이터의 신뢰성을 제공하고 사용자의 목적에 따라 모델을 수정할 할 수 있어 세부적인 설정이 가능하다. 그러나 OPNET이 제공한 SDN 모델의 출시가 2016년으로 이를 활용한 SDN M&S 구축사례 연구가 존재하지 않는다.

본 논문에서는 SDN M&S를 위한 Mininet과 OPNET 간 성능 비교 분석을 수행한다. 성능 비교를 위한 분석 모델을 제시한다. 효율적인 분석을 위하여 Mininet의 토폴로지 환경을 OPNET에서 동일하게 적용하기 위한 토폴로지 변환 방법을 제안한다. 또한 양 시뮬레이터에서의 트래픽 설정 방법을 설명한다. 이 모델을 통해 Mininet과 OPNET에서의 성능 지표에 대한 비교를 수행하고 이를 분석한다.

논문의 구성은 다음과 같다. 2장에서는 SDN 시뮬레이터로 사용할 Mininet과 OPNET에 대하여 살펴보고, 3장에서는 시뮬레이터간의 비교 분석 모델을 제시한다. 4장에서는 제시한 모델을 활용하여 Mininet과 OPNET의 성능지표를 비교하고 그 결과를 분석한다. 마지막으로 5장에서는 결론을 맺는다.

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM

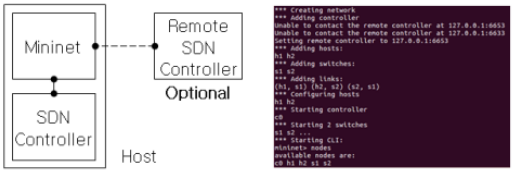
2. SDN M&S 도구

2.1 Mininet

Fs-SDN[6], NS-3[7], EstiNet[8] 등 SDN을 위한 다양한 시뮬레이션 도구가 존재하지만 Mininet은 가장 널리 사용되는 네트워크 에뮬레이터이자 시뮬레이터이다. OS-레벨의 가상화 기능을 이용해 빠른 시뮬레이션 속도를 제공하며 이에 확장성 측면에서도 강점이 있다. 또한 OpenFlow 프로토콜을 완벽하게 지원하는 시뮬레이터이며 다양한 SDN 스위치 장비와 연동된다. 현재 공개된 여러 SDN 오픈소스들과 뛰어난 호환성을 보여준다. 이러한 장점으로 대다수의 SDN 연구에서는 Mininet을 사용해왔다.

하지만 실제 환경과 다양한 시나리오를 모사하기에는 모델의 다양성이 부족하다. Mininet 버전 2.2.2은 SDN 스위치, 호스트 외에도 일반 스위치와 일반 라우터 모델(Legacy Model)만을 제공한다. 링크 모델은 1개를 제공하는데 링크 대역폭, 지연, 손실 확률의 세부 파라미터에 대한 설정만이 가능하다. 또한, 자체적인 트래픽 모델과 성능분석 기능이 없다. 대신 Ping, Iperf[9], D-ITG[10]같은 외부 트래픽 생성 도구로 트래픽을 발생해야 하고 성능분석은 패킷을 직접 수집해 분석하거나 다른 외부 도구를 사용해야 하는 단점이 있다. 따라서 네트워킹 시스템의 안정성과 신뢰성을 검증하는 실험, 현실적인 네트워크 환경이 필요한 실험 등에서 Mininet을 시뮬레이션 도구로 사용하기엔 부족한 면이 있다.

(그림 1) (a)는 Mininet의 구성을 보여준다. 일반적으로 Mininet은 한 호스트에서 가상의 스위치, 호스트를 구성한다. SDN 컨트롤러도 같은 호스트에 위치할 수 있으나, controller 옵션을 통해 외부의 컨트롤러와 연동할 수 있다. Mininet은 (그림 1) (b)와 같이 커맨드라인 인터페이스에서 동작하며, 실행 시 호스트와 스위치를 구성하고 이를 가상의 링크를 통해 연결한다. 시뮬레이션이 시작되면 xterm 명령어를 통해 각 노드의 터미널로 접근하여 리눅스 운영체제와 동일한 명령을 사용할 수 있다.



(a) Mininet 구성 (b) Mininet CLI

(그림 1) Mininet의 구성과 인터페이스

2.2 OPNET

OPNET Modeler은 네트워크 시뮬레이터로 다양한 모델을 포함하고 있으며 이번 버전 18.6부터 SDN을 지원하기 위한 OpenFlow 기반 스위치 모델을 제공한다. OpenFlow 스위치 모델은 OpenFlow v1.3의 명세서를 바탕으로 구현되었으며 외부 실 장비와 M&S를 지원하는 SITL(System-In-The-Loop) 모듈을 통해 외부 SDN 컨트롤러와 연결한다. 이에 따라 SDN 컨트롤러 상관없이 OpenFlow 1.3을 지원한다면 OPNET의 환경과 상호작용할 수 있으나 현재는 OpenDayLight[11]와 함께 구동할 때 가장 안정적으로 시뮬레이션이 가능하다.

OPNET은 기본적으로 다양한 실제 장비 모델을 벤더 별로 제공하고 있으며 실제 명세에 맞게 설계되어 있다. 노드 모델들은 PHY 계층부터 APP 계층을 포함하여, 실제 환경과 유사한 네트워크를 구성할 수 있고 각각의 성능지표를 수집할 수 있어 성능분석에 유리하다.

또한 OPNET은 견고한 트래픽 모델을 제공한다. 정밀한 서비스 트래픽을 모사하기 위해 Profile, Application, Task, Phase를 세부적으로 설정할 수 있고, 일반 트래픽 생성 도구와 같이 간단하게 설정할 수도 있다. 또한 백그라운드 트래픽을 설정하여 단대단 성능지표를 수집할 필요가 없는 유통 트래픽을 논리적으로 흐르게 하여 시뮬레이션 부하 및 소요시간을 줄일 수 있다.

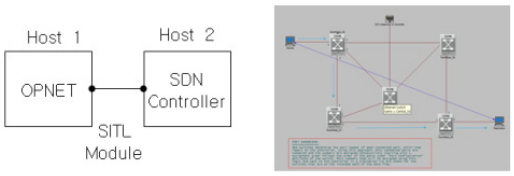
추가로 여러 가지 시나리오에 반영할 수 있는

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM

Configuration 모델이 존재하며 이를 이용한 다양한 연구가 진행되어 왔다. 예를 들어, 노드의 경로를 설정하는 이동성 모델을 활용한 연구[12], 악성 트래픽을 발생하고 치료할 수 있는 CyberEffect 모델을 통한 DDoS 공격 및 탐지 M&S 연구 [13] 등이 있다. 이를 이용한다면 SDN 환경에 다양한 시나리오를 시험해 볼 수 있다.

하지만 OPNET의 SDN 모델은 다음의 기능을 포함하고 있지 않다. 스위치와 컨트롤러 간의 논리적 포트, 하나의 스위치가 OpenFlow와 기존 라우팅을 혼용하여 사용하는 하이브리드 스위칭, IP 파편화 재조립, 다중 컨트롤러 기능이다.

(그림 2)의 (a)는 OPNET의 SDN 환경 구성을 나타낸다. Mininet과 달리, OPNET은 SITL 모듈을 통해 외부의 컨트롤러와만 통신할 수 있다. 즉, OPNET의 SDN 시뮬레이션을 위해서 2대의 장치가 필요하다. OPNET은 (그림 2) (b)와 같이 GUI 기반의 세뮬레이터이다. 토폴로지에는 SITL 모듈을 지원하는 SITL 노드/링크를 반드시 포함해야 하며 트래픽과 같은 시나리오 설정은 시뮬레이션 이전에 정의해야 한다.



(a) OPNET SDN 구성 (b) OPNET GUI

(그림 2) OPNET SDN의 구성과 인터페이스

3. Mininet과 OPNET 간 성능 비교 분석 모델

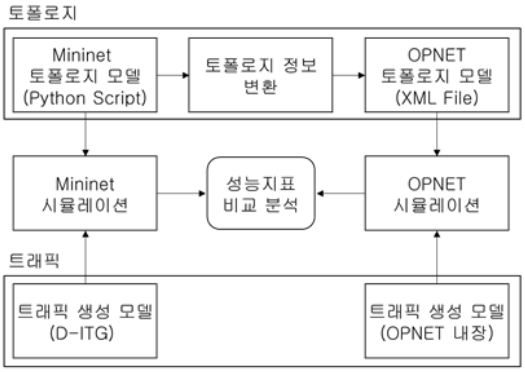
3.1 분석 모델 구성

시뮬레이터 간의 성능 비교를 위한 분석 모델의 구성은 (그림 3)과 같다. 네트워크 시뮬레이션에서의 일반적인 시나리오 구성은 토폴로지와 트래픽으로 구성된다. 본 논문에서 설명하는 토폴로지는 네트워크 토폴로지를 말하며 네트워크 장비인 노드들이 링크를 통해 물리적으로 연결되어 있는 데이터 통신망의 상태나 모양을 의미한다.

동일한 실험 환경을 만들기 위한 첫 번째 단계는 동일한 토폴로지를 각 시뮬레이터에 구성하는 것이다. Mininet에서 다양한 실험이 진행되어 왔으므로, Mininet의 모의실험을 위한 토폴로지 모델을 OPNET의 토폴로지 모델에서 사용할 수 있도록 변환하는 작업을 수행한다.

두 번째는 트래픽 모델로 시뮬레이터마다 동일한 소스노드에서 동일한 목적지노드에 데이터가 전달되어야 한다. OPNET의 트래픽 모델은 시작시간, 종료시간, 발생 빈도, 패킷 크기 등의 변수를 확률분포로 지정하여 생성하는 방식이다. Mininet의 경우 확률분포 기반 트래픽 생성을 지원하는 외부 도구인 D-ITG를 사용한다면 OPNET과 유사한 분포를 갖는 트래픽을 네트워크에 유통할 수 있다.

마지막으로, 토폴로지 모델과 트래픽 모델을 각 시뮬레이터에서 IMPORT하여 시뮬레이션을 수행해 그 결과를 비교한다.



(그림 3) 성능 비교 분석 모델 구성

3.2 토폴로지 정보 변환

외부로부터 토폴로지 모델을 IMPORT할 때, Mininet은 파이썬 스크립트를, OPNET은 XML 포맷을 사용한다. 따라서 기존 SDN의 M&S를 위해 구성한 Mininet의 토폴로지를 OPNET에서 사용할

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM

수 있도록 변환한다면, 시간을 단축할 수 있고 더 다양한 시나리오를 대상으로 시뮬레이션을 수행할 수 있다.

(그림 4)는 토폴로지 정보가 담긴 Mininet의 파이썬 스크립트의 예시이다. 첫째 줄의 from mininet.topo import Topo 는 토폴로지 정보로 사용하기 위한 Mininet의 토폴로지 모듈을 가져오는 코드이다. class SDNTopo(topo):는 토폴로지를 위한 클래스를 선언하기 위한 것이며 해당 클래스의 이름은 자유롭게 변경할 수 있다. def __init__(self): 와 Topo.__init__(self)은 해당 클래스의 초기화 메서드로 Mininet에서 실행 시 토폴로지가 실행되기 위함이다. 노드 정보 중 호스트 정보를 추가하기 위하여 self.addHost() 메서드를 사용한다. 그림 3의 예에 따르면 h1의 호스트를 상기 메소드 명령을 통해 추가하였다. 노드 정보 중 스위치 정보를 추가하기 위하여 self.addSwitch() 메서드를 사용하며 예에 따르면 s1의 스위치를 상기 메소드 명령을 통해 추가하였다. 링크 정보를 추가하기 위하여 self.addLink() 메서드를 사용한다. 예에 따르면 h1과 s1간의 링크를 연결하였다. 마지막으로 lamda 함수를 통해 해당 클래스를 호출한다.

```
<subnet name = "topo">
     <node name="h1" model ="wkstn">
          <attr name = "IP Address" value="10.0.0.1"/>
          …
     </node>
     <node name="s1" model = "of_switch_eth4">
          …
     </node>
     …
     <link name="h1<->s1" model="100BaseT"
      srcName="h1" dstName="s1">
          …
     </link>
     …
</subnet>
```

(그림 4) Mininet 토폴로지 모델 (파이썬 스크립트)

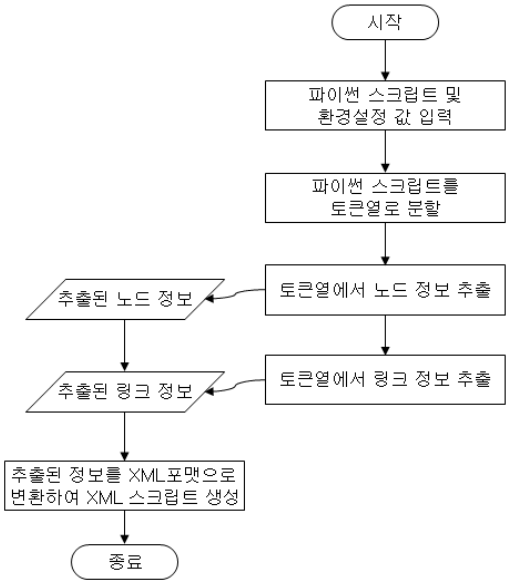
(그림 5)는 토폴로지 정보가 담긴 Opnet 시뮬레이터의 XML문서의 예시이다. 먼저 토폴로지를 정의하는 서브넷 정보는 엘리먼트 <subnet>으로 정의되며 속성 name이 정의된다. 노드 정보는 엘리먼트 <node>로 정의되며, 기본 속성은 name과 model로 구성되며, 추가 세부 속성으로 IP 주소, MAC 주소, CPU를 포함할 수 있다. 그림 2의 예시에서 노드 h1의 모델은 wkstn으로 워크스테이션을 의미하며 Opnet에 존재하는 여러 장치 모델 중 하나이다. 추가 세부 속성으로 IP주소의 값은 10.0.0.1로 설정되어 있다. 예시에서 노드 s1의 모델은 of_switch_eth4로 4개의 이더넷 포트를 갖는 오픈플로우 스위치이다. 링크 정보는 엘리먼트 <link>로 정의되며, 기본 속성은 name, srcName, dstName으로 구성된다. 링크 정보는 반드시 2개의 노드 사이를 연결하기 위하여 srcName 와 dstName을 갖고 있어야 한다. 링크 정보의 기본 속성은 종단 노드들의 이름을 포함하며 세부 속성으로 대역폭, 지연시간, 손실을 하나 이상 포함할 수 있다.

```
from mininet.topo import Topo
class SDNTopo(Topo):
     def __init__(self):
        Topo.__init__(self)
        h1 = self.addHost('h1')
        …
        s1 = self.addSwitch('s1')
        …
        self.addLink(h1,s1)
        …
topos = {'mytopo':(lambda:SDNProjectTopo())}
```

(그림 5) OPNET 토폴로지 모델 (XML)

(그림 6)은 변환 방법의 순서도이다. 먼저 토폴로지 정보가 포함된 파이썬 스크립트 및 환경설정값을 입력받는다. 일반적으로 토폴로지 정보에는 노드 정보, 링크 정보가 포함된다. 또한, 환경설정값은 시나리오의 이름, 시나리오의 망 규모, SDN 컨트롤러의 IP 주소 및 SDN 컨트롤러와 연결된 어댑터의 정보 등을 의미한다.

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM



(그림 6) 토폴로지 정보 변환 방법 순서도

다음으로 입력한 파이썬 스크립트를 토큰열로 분할한다. 이 단계에선 파이썬 스크립트에 반복문이 존재하는지 여부를 판단한다. 반복문이란 파이썬 스크립트에서 사용되는 for, while 등의 명령어로 반복문이 존재하는 경우, 이를 처리한 후 토큰열 분할을 진행한다. 그 이후 파이썬 스크립트 각 줄을 반환 변수, 함수 및 매개 변수로 분할하고 이 분할된 토큰열을 메모리에 저장한다. (그림 4)의 예를 들면, "h1 = self.addHost('h1')"을 토큰열로 분할하는 경우, "h1"을 반환 변수, "self.addHost"를 함수, "('h1')"의 h1을 함수의 매개 변수로 분할된다.

다음, 토큰열에서 노드 정보를 추출한다. 먼저 토큰열을 분석하여 "addHost" 함수 및 "addSwitch" 함수를 포함하는 토큰열 구조체를 탐색한 후, "addHost"를 포함하는 경우를 호스트 노드, 토큰열로 "addSwitch"를 포함하는 경우를 스위치 노드로 결정한다. 또한, "addHost" 함수 및 "addSwitch" 함수의 첫 번째 매개 변수는 노드의 이름에 해당하므로, 첫 번째 매개 변수를 호스트 노드 및 스위치 노드의 이름으로 설정한다. 스위치 노드들에 대해서는, 환경설정 값으로 수신한 SDN 컨트롤러의 IP 주소 등을 노드 정보에 추가한다. 다음으로 스위치 노드들 및 호스트 노드들의 속성 파라미터 및 속성 값을 노드 정보로 추출할 수 있다. Mininet에서 설정 가능한 속성 파라미터는 IP 주소, MAC 주소 및 CPU 크기 등이다. 속성 파라미터가 토큰열에 포함되는 경우, 스크립트 변환 장치는 해당 속성 파라미터 및 이에 대응되는 속성 값을 스위치 노드들 또는 호스트 노드들의 노드 정보로 추출한다.

링크 정보 추출 단계에서는 토큰열을 분석하여 링크의 양 종단 노드를 결정할 수 있다. 먼저 "addLink" 함수를 토큰열로 포함하는 토큰열 구조체를 탐색한다. "addLink" 함수의 경우 기본 속성값으로 2개의 매개변수를 가지며 이는 양 종단 노드를 의미한다. "addLink" 함수는 추가적인 속성 파라미터 및 이에 대응되는 속성 값으로 예를 들어, 대역폭 "bw=10", 지연시간 "delay=5ms" 등을 포함할 수 있다. 다음으로 종단 노드가 토폴로지에 포함되는지 여부를 판단해야 한다. 먼저 토폴로지에 포함되는 노드들을 노드 정보 추출단계에서 식별하였으므로 가능하다. 마지막으로 링크의 속성 파라미터 및 속성 값을 링크 정보로 추출한다.

마지막으로, 노드 정보 및 링크 정보를 XML 포맷으로 변환함으로써 XML 스크립트를 생성할 수 있다.노드의 정보를 XML 포맷으로 변환하기 위하여 엘리먼트 <node>에 이름을 name= ""으로, 모델 정보를 model= ""과 같이 추가한다. 호스트 모델은 ethernet_wkstn_adv, ethernet_Server_adv 등을 의미하며 필요에 따라 다른 모델로 사전 설정이 가능하다. 스위치 모델은 이더넷 인터페이스인 개수에 따라, of_switch_eth4_adv, of_switch_eth16_adv으로 구성한다. 추가적인 속성 파라미터는 엘리먼트 <node>의 하위 항목으로 <attr name = "" value = ""/>와 같이 변환된다. 노드 정보가 남아있는 경우, 처음으로 돌아가 위 과정을 반복 수행한다. 추가적으로, SDN 컨트롤러 노드를 XML 포맷으로 변환하여 XML 스크립트에 추가한다. SDN 컨트롤러의 모델 정보는 SITL 노드 모델 정보로

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM

sitl_virtual_gateway_to_real_world로 설정된다.

링크의 정보를 XML 포맷으로 변환하기 위하여 엘리먼트 <link>에 이름을 name= “”으로, 모델을 model= “”과 같이 추가할 수 있다. 한편, name에는 “h1<->s1”과 같이 종단 노드의 이름 사이에 <->를 입력한다. 링크 모델은 기본적으로 100baseT으로 필요에 따라 사전에 설정이 가능하다. 또한 srcNode, destNode에 각 종단 노드를 추가한다. 종단 노드의 정보는 엘리먼트 <link>의 하위 항목으로 <attr name = “” value = “”/>와 같이 변환될 수 있다. 추가적인 속성 파라미터 또한 엘리먼트 <link>의 하위 항목으로 <attr name = “” value = “”/>와 같이 변환될 수 있다. 다른 링크 정보가 남아있는 경우, 단계 810으로 돌아가 상술한 과정을 반복 수행할 수 있다. 추가적으로 SDN 컨트롤러로 연결되는 SITL 링크 또한 SDN 스위치와 연결해 주어야한다. 해당 링크의 모델 정보는 sitl_virtual_eth_link이며, sdn스위치와 SITL 노드와 연결해야 한다.

3.3 트래픽 생성 모델

2장에서 언급하였듯이, Mininet은 Ping, Iperf, D-ITG와 같은 외부의 트래픽 생성 도구를 통해 호스트 노드에서 트래픽을 생성할 수 있다. 일반적인 실험에서는 리눅스에서 기본적으로 사용할 수 있는 Ping과 Mininet과 함께 설치되는 Iperf가 주로 사용된다. 그러나 본 논문에서는 D-ITG를 사용한다. 다른 트래픽 생성 도구와 다르게 확률 분포 기반의 트래픽 생성이 가능하며 이는 OPNET의 트래픽 생성 모델의 방식과 같다. 또한 로그파일과 분석 도구를 제공하여 손쉽게 원하는 데이터를 얻을 수 있다.

OPNET의 트래픽 모델은 Profile, Application, Task, Phase에 맞추어 견고하게 설정할 수 있으나 Application Demand 모델을 이용한다면, D-ITG와 같이 트래픽 발생에 필요한 기본 변수들만을 설정해 트래픽을 생성할 수 있다.

4. 실험 결과

4.1 실험 환경

OPNET에서 SDN 모델을 사용하기 위해선 SITL(System-In-The-Loop)이라는 모듈이 추가적으로 필요하다. 이는 외부 실 장비와 연결하여 M&S를 지원하는 모듈로 외부 SDN 컨트롤러와 연결할 때 사용한다. 외부 컨트롤러는 기존의 SDN 컨트롤러와 같이 외부의 장치에 구동한다. 본 실험에서는 OPNET 18.7을 Windows 10에 설치하였으며, SITL 모듈을 통해 Ethernet Cable로 Linux PC(Ubuntu 16.04.4)와 연결하였다. Linux PC에는 SDN 컨트롤러로 OpenDaylight Nigtrogen SR2을 설치하였다. Mininet 환경은 Mininet 2.2.2와 OpenDaylight를 동일한 Linux PC에 함께 설치하였다.

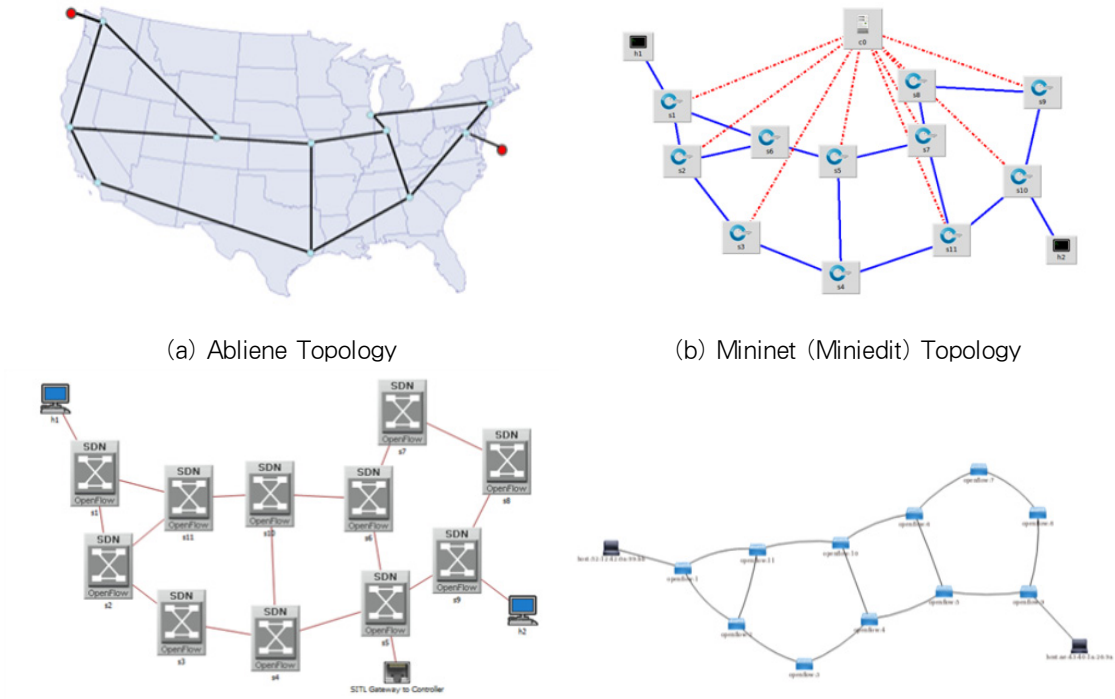
4.2 성능 지표 비교 및 분석

1) 토폴로지 정보 변환 검증

성능 지표를 비교하기에 앞서 Mininet으로 구성한 토폴로지의 정보를 OPNET의 토폴로지 정보로 변환한 후 동일하게 구성되었는지를 확인한다. 토폴로지 변환 방법을 검증하기 위한 예시 토폴로지는 Abliene 망[14]으로 (그림 7)의 (a)와 같이 구성하였다. Abliene은 90년대 말까지 구성된 미국의 대학, 회사, 정부 연구소를 연결하는 백본 네트워크로 현재 Internet2 네트워크의 전신이며 다양한 네트워크 실험에서 사용되는 토폴로지이다. SDN 관련 연구에서도 사용되었다[15][16]. 총 11개의 백본 노드와 14개의 링크를 포함하며, 백본 노드를 OpenFlow 스위치로 대체하여 구성하였다. 추가로 패킷을 주고받는 2개의 호스트를 2개의 링크로 연결하였다

(그림 7)의 (b)는 구성한 Mininet 토폴로지 정보를 Mininet의 GUI 도구인 Miniedit으로 표현한 것이다. 이 토폴로지 정보를 파이썬 스크립트로 export하여 토폴로지 변환 방법에 따라 OPNET의 토폴로지 정보로 변환한다. (그림 7)의 (c)는 O

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM



(a) Abliene Topology (b) Mininet (Miniedit) Topology

(c) OPNET Topology (d) Topology from SDN Controller

(그림 7) 토폴로지 정보 변환 검증

PNET의 토폴로지 정보로 변환된 XML을 OPNET에 IMPORT한 것으로 (그림 7)의 (a)와 유사하게 배치하였다. Mininet의 토폴로지 정보는 GUI를 위한 위치정보가 없으므로 Import시 임의의 Ring의 형태로 배치된다. (그림 7)의 (d)는 OPNET의 시뮬레이션을 수행하여, 외부 컨트롤러인 OpenDaylight의 Topology View로 확인한 것이다. 이 결과로 미루어 보아, 제안하는 방법을 통한 토폴로지 변환이 제대로 이루어 졌음을 시각적으로 확인하였다.

2) 시뮬레이션 시나리오 구성

토폴로지는 선형 토폴로지로 설정한다. 메시, 트리형과 같은 다른 토폴로지의 실험은 구조보다는 전송 경로에 위치한 스위치의 대수와 연관되므로 선형 토폴로지에서 중계 스위치를 늘려가며 실험을 진행한다. 각 스위치들은 직렬로 연결되어 있으며 호스트는 양 종단의 스위치와 연결되어 있다. 개수를 5개, 10개, 15개로 증가시키고 이에 따른 성능 지표의 변화를 확인한다.

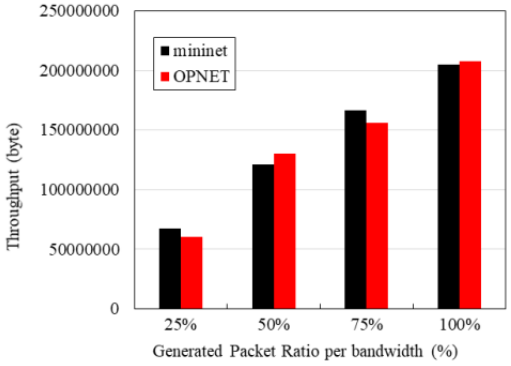
노드들을 연결한 링크 모델로 OPNET에서는 100BaseT 모델을 사용하였으며, Mininet에서 대역폭을 100Mbps로 설정하였다.

네트워크에 유통될 트래픽 생성 모델로 Mininet에서는 D-ITG를 이용하였고 OPNET에서는 내장된 트래픽 발생 모델인 Application Demand 모델을 사용하였다. 각 패킷 생성 모델이 다른 Random Seed를 사용하므로 완벽하게 일치하는 패킷을 생성할 수 없으나 동일한 확률분포 모델을 따르므로 유사한 경향으로 발생한다. 이 실험에서는 평균 1024 Byte인 지수분포를 따르는 UDP 패킷을 30초동안 발생시키도록 설정하였다. 패킷의 발생량 역시 지수분포를 따르며 링크의 용량인 100Mbps에 맞추어 초당 2500개(25%), 5000개(50%), 7500개(75%), 10000개(100%)로 증가시키며 변화를 확인한다.

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM

3) **처리량** (Throughput)

(그림 8)은 Mininet과 OPNET간 처리량 비교한 결과이다. 처리량은 오직 노드 15대에서 초당 패킷 발생량을 변화해가며 측정하였다. 패킷 발생량과 사이즈가 모두 지수 분포를 따르기 때문에 일치하지는 않으나 유사한 처리량을 보임을 확인할 수 있다. 이에 따라 Mininet과 OPNET의 트래픽 발생 모델이 동일한 분포로 패킷을 발생하고 있음을 확인할 수 있다.



(그림 8) 처리량 성능 비교

4) **단대단 지연시간** (End-To-End Delay)

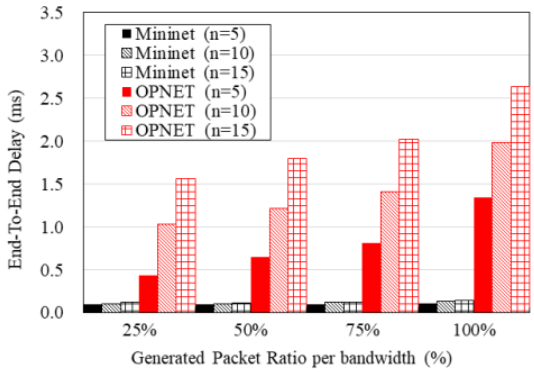
(그림 9)는 Mininet과 OPNET간의 단대단 지연시간을 비교한 실험 결과이다. (그림 9) (a)에서, Mininet은 중계 스위치의 대수를 5대에서 15대로 증가시켜도 단대단 지연시간은 거의 차이가 없다. OPNET의 지연시간은 대수가 늘어날수록 선형적으로 비례하게 증가하는 것을 확인 할 수 있다. 25%, 50%, 75%, 100% 모든 패킷의 초당 발생량에서 같은 경향을 나타낸다. (그림 9) (b)에서, 초당 패킷 발생량을 25%에서 100%로 증가시킴에도 Mininet의 단대단 지연시간은 거의 차이가 없다. 앞의 결과와 마찬가지로, OPNET은 패킷의 발생량이 증가해 링크 사용량이 상승할수록, 지연시간이 점차 커짐을 확인 할 수 있다.

5) **지터** (Jitter)

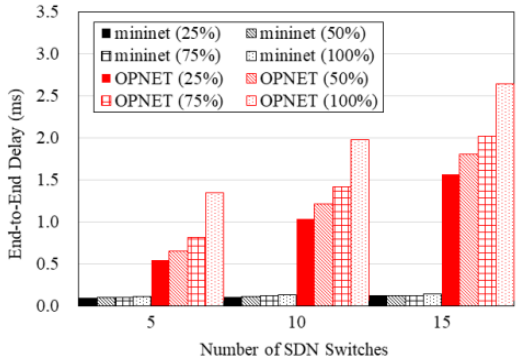
(그림 10)은 Mininet과 OPNET 간의 지터를 비교한 실험 결과이다. Jitter는 단대단 지연시간의 변화 내지 차이를 의미한다. (그림 10) (a)에서, Mininet은 중계 스위치의 대수를 증가시켜도 차이가 없다. OPNET은 대수를 증가시킬수록 지터가 증가한다. (그림 10) (b)에서도 마찬가지로, 초당 패킷 발생량을 증가시켜도 Mininet은 큰 변화가 없다. OPNET은 링크 최대 가용량에 도달하기 전까지는 점점 감소하다가 최대 가용량에 도달하면 급격하게 상승한다.

4.3 결과 분석

토폴로지 정보 변환 방법의 검증을 통해 동일한

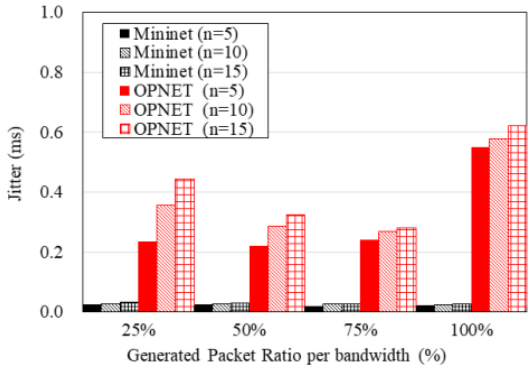


(a) 스위치 대수 별

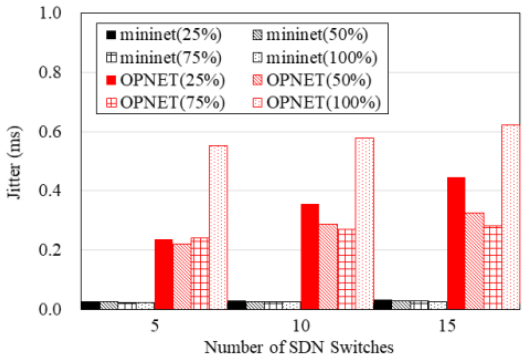


(b) 링크 용량에 따른 초당 패킷 발생량 별

(그림 9) 단대단 지연시간 성능 비교



(a) 스위치 대수 별



(b) 링크 용량에 따른 초당 패킷 발생량 별

(그림 10) 지터 성능 비교

토폴로지가 생성되고 처리량 비교에서 유사한 양의 트래픽이 SDN 스위치를 통해 정상적으로 유통됨을 확인하였다. 따라서 두 시뮬레이터 간의 비교 분석 모델이 의도대로 동작함을 검증하였다.

또한 이 모델을 기반으로 단대단 지연시간과 지터를 측정하여 비교하였다. 중계 스위치와 트래픽 발생의 증감에도 Mininet의 단대단 지연시간과 지터가 일정한 것을 미루어 볼 때, 가상의 Mininet 환경은 네트워크 부하를 반영할 수 없음을 확인하였다. 반대로 OPNET은 링크, 노드의 트래픽 처리시 노드 수와 패킷 발생량에 따라 변동하는 지연시간과 지터를 확인할 수 있었다. 따라서 실 환경을 반영하기 위한 시뮬레이션이나 망의 안정성, 신뢰성의 실험에 있어서는 Mininet보다 OPNET을 사용하는 것이 바람직하다고 할 수 있다.

5. 결론

기존의 SDN M&S에서는 Mininet을 중심으로 진행되어 왔다. Mininet은 간단한 환경을 빠르게 실험해보는 경우에는 유리하지만, 제공하는 노드 모델과 링크 모델이 단순하고 설정할 수 있는 속성 파라미터가 적기 때문에 실험의 신뢰성을 보이기가 어렵다. 본 논문에서는 SDN M&S를 위한 Mininet과 OPNET 간 성능 비교 분석을 수행하였다. 효율적인 분석을 위하여 토폴로지 정보 변환과 트래픽 모델을 포함하는 시뮬레이터간 비교 분석 모델을 제안하였다. 이를 바탕으로 Mininet과 OPNET에서 처리량, 단대단 지연시간, 지터에 대한 비교를 수행하고 분석하였다.

분석 결과에 따르면, 점차 SDN의 적용 범위가 확대되고 구성 요소들의 다양한 세부 특성이 반영되면, Mininet에서 실험한 연구들은 실환경과 상이한 결과를 나타내어 신뢰성을 잃을 수 있다. 따라서, 실환경을 고려하거나 안정성, 신뢰성 등을 평가하는 실험을 수행할 시 OPNET을 이용한다면 더욱 신뢰할 수 있는 환경에서 M&S를 수행할 수 있을 것으로 보인다.

▌감사의 글

본 연구는 방위사업청과 국방과학연구소가 지원하는 미래전투체계 네트워크 기술 특화연구센터 사업의 일환으로 수행되었습니다.(UD160070BD)

▌참고문헌

[1] O. N. Fundation, "Software-defined networking: The new norm for networks," ONF white Paper, 2012.

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM

〔2〕 N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, J. Turner, "OpenFlow: enabling innovation in campus networks," ACM SIGCOMM Computer Communication Review, vol. 38, no. 2, pp. 69-74, 2008.

〔3〕 B. Lantz, B. Heller, N. McKeown, "A Network in a Laptop: Rapid Prototyping for Software-Defined Networks," 9th ACM Workshop on Hot Topics in Networks, Oct. 2010.

〔4〕 OPNET, Available: http://www.riverbed.com/

〔5〕 M. Azizi, B. Redouane Benaini, B. Mouad, "Delay measurement in OpenFlow-enabled MPLS-TP network," Modern Applied Science, Vol.9 No.3, 2015.

〔6〕 M. Gupta, J. Sommers, P. Barford, "Fast accurate simulation for SDN prototyping," 2nd ACM SIGCOMM workshop on Hot topics in software defined networking, pp. 31-36, 2013

〔7〕 T. Henderson, M. Lacage, G. Riley, M. Watrous, G. Carneiro, T. Pecorella, "Network Simulator 3," Available: https://www.nsnam.org

〔8〕 S. Wang, C. Chou, C. Yang, "EstiNet OpenFlow network simulator and emulator," IEEE Communications Magazine, vol. 51, no. 9, pp. 110-117, 2013.

〔9〕 A. Tirumala, F. Qin, J. Dugan, J. Ferguson, K. Gibbs, "iPerf: TCP/UDP Bandwidth Measurement Tool," http://iperf.fr, 2005

〔10〕 A. Botta, A. Dainotti, A. Pescapè, "A tool for the Generation of Realistic Network Workload for Emerging Networking Scenarios," Computer Networks, Vol. 56, No. 15, pp. 3531-3547, 2012.

〔11〕 OpenDaylight, Available: https://www.opendaylight.org

〔12〕 김학원, 김광수, 윤병성, 노병희, "Bloom Filter를 이용한 AODV 라우팅 성능 향상 방안," 한국차세대컴퓨팅학회 논문지, Vol. 10, No.3, pp.52-61, 2014. 6.

〔13〕 이병욱, 이철웅, 신승훈, 노병희, 안명길, 김용현, "OPNET의 Cyber Effect 모델을 활용한 유선 전술 통신망의 DDoS 공격 및 탐지 모델링 및 시뮬레이션 구현," 한국차세대컴퓨팅학회 논문지 Vol. 12, No. 1, pp. 75-83, 2016. 2.

〔14〕 SNDlib. The Abilene WAN Topology Available: http://sndlib.zib.de/home.action

〔15〕 J. Jiang, H. Huang, J. Liao, S. Chen, "Extending Dijkstra's Shortest Path Algorithm for Software Defined Networking", IEEE 16th Asia-Pacific Network Operations and Management Symposium, Sep. 2014.

〔16〕 Y. Guo, Z. Wang, X. Yin, X. Shi, J. Wu, "Traffic engineering in hybrid SDN networks with multiple traffic matrices," Computer Networks, Vol. 126, pp. 187-199, Oct. 2017.

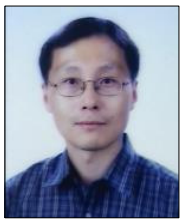
〔17〕 D. Suh, S. Pack, "Low-Complexity Master Controller Assignment in Distributed SDN Controller Environments," IEEE Communication Letters, Vol. 22, No. 3, Mar. 2018.

저자소개

◆ 이승운

- 2015년 2월 아주대학교 정보컴퓨터공학과(공학사)
- 2017년 2월 아주대학교 소프트웨어특성화학과(공학석사)
- 2018년 3월~현재 아주대학교 컴퓨터공학과 박사과정
- 관심분야: 미래인터넷, 네트워크 보안, 군방전술통신 네트워크

◆ 노병희



- 1987년 2월 한양대학교 전자공학과(공학사)
- 1989년 2월 한국과학기술원 전기및전자공학과(공학석사)
- 1998년 2월 한국과학기술원 전기및전자공학과(공학박사)
- 1989년 3월~1994년 2월 한국통신 통신망연구소
- 1998년 2월~2000월 3월 삼성전자
- 2014년 3월~2015년 2월 국방과학연구소 객원연구원
- 2000년 3월~현재 아주대학교 소프트웨어학과 교수
- 관심분야: 유/무선 멀티미디어 통신 응용, 트래픽 제어, IoT 플랫폼 SW, 미래 인터넷 기술, 국방전술통신 네트워크, 네트워크 보안

[Provider:earticle] Download by IP 203.232.XXX.XX1 at Monday, April 6, 2026 9:54 AM